GRUPPE A: 22.10.2026 GRUPPE B: 29.10.2026

E 308 – Programmieren 3

Übung zu Termin 3: Daily & Klassen im Projekt

D. Wild

Hochschule Kempten – Fakultät Elektrotechnik

Agenda

Daily

Sprint-Themen heute

Beispiel: Datenzugriff als Klasse

Ablauf heute

90 Minuten – Daily reihum, die anderen Teams arbeiten weiter

Minute	Daily mit dem Dozenten	Alle anderen Teams
0–10	Team 1 bzw. 4	Thema 1: Codestruktur überlegen
10–20	Team 2 bzw. 5	Thema 1: Codestruktur überlegen
20–30	Team 3 bzw. 6	Thema 1 bzw. Thema 2
30–80	Themen 2–3 – Betreuung an den Tischen	
80–90	Pull Request & Review, Board aktualisieren	

Gruppe B am 29.10. — Sprint 1 endet am 28.10. – statt des Dailys beginnt die Übung mit dem ersten gemeinsamen Sprintwechsel (ca. 45 Minuten), danach Themen 2–3.

Das Daily

10 Minuten pro Team, am Board

Jede Person beantwortet kurz:

1. Was habe ich seit dem letzten Treffen geschafft?
2. Was mache ich als Nächstes?
3. Was hindert mich gerade?

Danach gemeinsam am Board:

- Stehen alle Stories in der richtigen Spalte?
- Hat jede Story eine verantwortliche Person?

Spielregeln —

- GitHub-Projekt ist offen
- Streng 10 Minuten
- Probleme nur *nennen*, lösen danach in kleiner Runde

Reihenfolge — Gruppe A: Team 1, 2, 3
Gruppe B: Team 4, 5, 6

Die drei Themen im Überblick

Sprint 1 ist noch vorgegeben – arbeiten Sie die Themen der Reihe nach ab

	Thema	Ergebnis
1	Codestruktur im Team überlegen: Welche Klassen brauchen wir für Datenzugriffe etc.?	Skizze: Klassen & Module
2	`Application`-Klasse als **Projekt-Information** (Pflicht für alle Teams): Name, Version, Team, Maintainer, Tags; Methode `status()`	`application.py` mit Ihren Projektdaten
3	Projektstruktur (Paket, `__init__.py`, `app.py`) per Pull Request anlegen – jede Klasse im eigenen Modul	gemergter Pull Request

Beispiel für eine Datenzugriffsklasse (CSV lesen, `data.mean()`): Abschnitt „Beispiel“ am Ende.

Thema 1: Codestruktur überlegen

Erst skizzieren, dann programmieren – im ganzen Team

Leitfragen

- Woher kommen Ihre Daten? CSV, API, Simulation?
- Welche Klasse liest welche Daten? → *eine Klasse pro Datenquelle*
- Was soll eine Datenquelle können? (laden, Mittelwert, filtern, …)
- Wie heißt Ihr Paket? Welche Datei enthält welche Klasse?

Beispiel: Wetterkarte

```
weathermap/          <- Repository
|-- app.py
|-- requirements.txt
|-- data/
|   `-- kempten.csv
`-- weathermap/      <- Paket
    |-- __init__.py
    |-- application.py <- Application
    `-- measurements.py <- MeasurementSeries
```

`application.py`: Information, in allen Teams gleich.
`measurements.py`: Datenzugriff, je nach Projekt.

Ergebnis — Skizze auf Papier oder im Issue der Story – mit Klassennamen (englisch, PEP 8), Attributen und Methoden.

Thema 2: `Application` als Projekt-Information

```python
class Application:
    """Information des Projekts -- Pflicht für alle Teams."""

    def __init__(self, name, version, description, developers,
                 maintainer, tags):
        self.name = name
        self.version = version
        self.description = description
        self.developers = developers # Liste mit allen Teammitgliedern
        self.maintainer = maintainer # Ansprechperson für das Review
        self.tags = tags          # Stichworte: Themen, Frameworks

    def status(self):
        """Information als Text, z. B. für das Review."""
        return (f"{self.name} {self.version}: {self.description}\n"
                f"Team: {', '.join(self.developers)} "
                f"(Maintainer: {self.maintainer})\n"
                f"Tags: {', '.join(self.tags)}")
```

Die Projekte und Frameworks sind verschieden – der Information ist in allen Teams gleich aufgebaut

Thema 2: Ihre Information ausfüllen

Am Ende von <paket>/application.py

```
app_info = Application(
    name="Wetterkarte",
    version="0.1.0",
    description="Wetterdaten aus dem Allgäu "
                "auf einer Karte",
    developers=["A. Muster", "B. Beispiel",
                "C. Probe"],
    maintainer="A. Muster",
    tags=["streamlit", "folium", "wetter"],
)
```

- **Pflicht:** Datei `application.py` im Paket, genau diese Attributnamen
- **Version** mit jedem Sprint erhöhen: `0.1.0` nach Sprint 1, `0.2.0` nach Sprint 2, …
- **Tags**: Thema, Frameworks, Datenquellen
- `APP` wird überall importiert, z. B. `st.caption(APP.status())`

Thema 3: Projektstruktur per Pull Request

Jede Klasse in ihrem eigenen Modul

1. **Eine Person** legt die Struktur an: Branch `feature/projektstruktur`, Paket, `__init__.py`, `app.py`
2. Pull Request → Review → Merge
3. Alle anderen: `git pull` auf `main`, dann eigener Branch pro Klasse
4. Klassen aus dem Spike in ihr Modul verschieben

`app.py`

```python
from weathermap.application import APP
from weathermap.measurements import (
    MeasurementSeries)

print(APP.status())

kempten = MeasurementSeries("data/kempten.csv")
print(kempten.mean("temperature_c"))
```

Start aus dem Wurzelordner: `python app.py`

Konflikte vermeiden: Die Struktur zuerst mergen, *dann* parallel an den Klassen arbeiten – sonst ändern alle gleichzeitig dieselben Dateien.

Ausgangspunkt: Ihr Spike

Daten und Funktionen liegen getrennt

```python
# spikes/<vorname>/funktionen.py
def mittelwert(daten, feld):
    werte = [eintrag[feld] for eintrag in daten]
    return sum(werte) / len(werte)

print(mittelwert(MESSUNGEN, "temperatur")) # Daten bei jedem Aufruf mitgeben
```

Bisher

- Daten fest im Code (Liste von Dictionaries)
- Jede Funktion bekommt die Daten als Parameter

Heute

- Daten aus einer **CSV-Datei**
- Eine **Klasse** hält die Daten (Attribut) und die Auswertungen (Methoden) zusammen

Schritt 1: Eine Test-CSV anlegen

`data/kempten.csv`

```
date,temperature_c,humidity_pct
2026-10-12,11.4,78
2026-10-13,12.8,74
2026-10-14,10.2,85
2026-10-15,9.6,88
2026-10-16,13.1,70
```

- Erste Zeile: Spaltennamen, danach ein Datensatz pro Zeile
- Bewusst **klein**: Das Ergebnis lässt sich von Hand nachrechnen
- Mittelwert Temperatur: $(11.4+12.8+10.2+9.6+13.1)/5 = 11.42$

Warum eine Test-Datei? — Mit bekannten Werten sehen Sie sofort, ob Ihr Code richtig rechnet. Echte Daten kommen später dazu – die Klasse bleibt dieselbe.

Schritt 2: CSV mit pandas lesen

Erst ohne Klasse ausprobieren

```
python -m pip install pandas # einmal, in der aktivierten .venv
```

```
import pandas as pd

data = pd.read_csv("data/kempten.csv") # DataFrame = Tabelle
print(data.head())                     # die ersten Zeilen ansehen
print(data["temperature_c"].mean()) # eine Spalte -> 11.42
print(data.mean(numeric_only=True)) # alle Zahlenspalten
```

- `data["spalte"]` liefert eine Spalte, `.mean()` deren Mittelwert – ebenso `.min()`, `.max()`
- `pandas` in die `requirements.txt` eintragen

Schritt 3: Die Klasse

`weathermap/measurements.py`

```python
from pathlib import Path

import pandas as pd


class MeasurementSeries:
    """Eine Messreihe, die aus einer CSV-Datei gelesen wird."""

    def __init__(self, path):
        self.name = Path(path).stem # "data/kempten.csv" -> "kempten"
        self.data = pd.read_csv(path)

    def mean(self, column):
        """Mittelwert einer Spalte."""
        return self.data[column].mean()

    def means(self):
        """Mittelwerte aller Zahlenspalten."""
        return self.data.mean(numeric_only=True)
```

Was passiert in der Klasse?

Die Bausteine aus der Vorlesung

Code	Bedeutung
`__init__(self, path)`	liest die Datei *einmal* beim Erzeugen des Objekts
`self.data`	Instanzattribut: Jedes Objekt hat seine eigene Tabelle
`self.name`	aus dem Dateinamen abgeleitet: `Path(path).stem` macht aus `"data/kempten.csv"` den Namen `"kempten"`
`mean(self, column)`	Methode: arbeitet mit `self.data` – die Daten müssen nicht mehr übergeben werden
`means(self)`	`data.mean()` über alle Zahlenspalten

Vergleich zum Spike — `mittelwert(MESSUNGEN, "temperatur")` wird zu `kempten.mean("temperature_c")` – das Objekt kennt seine Daten selbst.

Schritt 4: Die Klasse benutzen

app.py im Wurzelordner

```python
from weathermap.measurements import (
    MeasurementSeries)

kempten = MeasurementSeries("data/kempten.csv")
print(kempten.name)
print(f"{kempten.mean('temperature_c'):.2f} °C")
print(kempten.means())
```

```
python app.py    # aus dem Wurzelordner!
```

Ausgabe

```
kempten
11.42 °C
temperature_c  11.42
humidity_pct   79.00
dtype: float64
```

11.42 – passt zur Handrechnung.

Projektstruktur wie in der Vorlesung: `weathermap/__init__.py`, `weathermap/measurements.py`, `data/`, `app.py`.

Typische Fehler

... und was dahintersteckt

Fehler	Ursache & Lösung
`data.mean()` ▷ `TypeError`	Die Spalte `date` ist Text und lässt sich nicht mitteln. Spalte wählen (`data["temperature_c"].mean()`) oder `data.mean(numeric_only=True)`
`FileNotFoundError: data/kempten.csv`	Aus dem falschen Ordner gestartet – der Pfad ist relativ zum Ordner, in dem das Terminal steht
`KeyError: 'temperatur'`	Spaltenname falsch geschrieben – `print(data.columns)` zeigt die echten Namen
`ModuleNotFoundError: No module named 'pandas'`	`.venv` nicht aktiviert oder pandas nicht installiert

Zusatz: mehrere Dateien mit `cls`

Eine Klassenmethode als zweiter Weg, Objekte zu erzeugen

```python
class MeasurementSeries:
    # ... __init__, mean, means wie bisher ...

    @classmethod
    def from_folder(cls, folder):
        """Lädt alle CSV-Dateien eines Ordners als Messreihen."""
        series = []
        for path in sorted(Path(folder).glob("*.csv")):
            series.append(cls(path))    # cls(...) = MeasurementSeries(...)
        return series


for s in MeasurementSeries.from_folder("data"):
    print(f"{s.name:<12} {s.mean('temperature_c'):5.2f} °C")
```

- Ausgabe: `kempten 11.42 °C`, `oberstdorf 8.30 °C`
- Es gibt noch kein Objekt, aus dem man lesen könnte – deshalb `cls` statt `self`